

1 AI Usage Log

This log records meaningful AI-assisted decisions used while building DesignGym. The final choices were reviewed, constrained, and implemented by the human author.

1.1 1. Fixed rubric schema over unstructured LLM prompts

Engineering decision: Rejected unstructured raw LLM prompts in favor of a fixed five-field rubric schema: **Criterion** → **Score** → **Evidence** → **Concern** → **Suggestion**. The implementation adds confidence and deterministic findings as separate fields.

Why: A raw narrative response is difficult to validate, compare across attempts, or render consistently. A fixed schema makes every review actionable, keeps evidence tied to the learner's own words, and enables structured JSON parsing with malformed-output rejection.

Implementation: `FeedbackItem`, `parseRubricEvaluation`, `DeterministicRubricEvaluator`, and the review UI enforce the contract. Scores are normalized to 0–100 and signals are derived from score thresholds.

1.2 2. Monolithic Attempt State Machine over microservices and queues

Engineering decision: Rejected complex microservices, queue, and worker designs suggested by AI in favor of a monolithic Attempt State Machine suitable for the two-day LLD boundary.

Why: The assignment evaluates product judgment and extensibility, not operational infrastructure. A small explicit state model is easier to reason about, test, and demonstrate while still preserving an extension seam for a future background evaluator.

Implementation: Attempt transitions only through `Draft` → `Submitted` → `Evaluating` → `Completed/Failed`. Illegal jumps throw `InvalidStateTransitionError`. The SQL-backed app can persist attempts without forcing queue infrastructure into the MVP.

1.3 3. Strategy Pattern over coupled submission code

Engineering decision: Refactored coupled submission handling into the Strategy Pattern to pass both change tests.

Why: Text, class-diagram, and future code submissions should not require edits to attempt lifecycle or evaluation orchestration. Likewise, a human, rule-based, or LLM evaluator should be replaceable without changing practice logic.

Implementation: `Submission` is the submission contract; `StructuredWrittenSubmission` and `DiagramSubmission` are concrete strategies. `EvaluatorStrategy` is the evaluator contract, and the fixture tests prove a replacement evaluator can be injected without changing the learner flow.

1.4 4. Async failure handling with deterministic fallback

Engineering decision: Implemented asynchronous failure handling with deterministic fallback instead of blocking synchronous LLM execution.

Why: LLM services can time out, return malformed JSON, or become unavailable. The learner should receive a useful review or a clear failed state rather than a crashed practice session.

Implementation: EvaluationEngine awaits the primary evaluator, catches timeout/failure, runs DeterministicRubricEvaluator, records source: "deterministic-fallback" and failure details, and transitions safely. If both evaluators fail, the attempt reaches Failed while preserving the submission.

1.5 Additional product decisions

DesignGym is positioned as a CipherSchools practice-and-feedback layer rather than a full LMS. The interface uses a quiet systems-lab visual language, structured evidence fields, persistent attempt history, and recurring weakness aggregation. Authentication, SQL persistence, account settings, password recovery, and contact verification were added as product infrastructure while keeping the domain evaluator boundary independent.